

First Come, First Served (FCFS)

FCFS serves processes in arrival order. Once a process starts, it keeps the CPU until it finishes.

1. Model

The visualizer uses one CPU and measures time in whole ticks. Processes only need CPU service; they do not block for I/O. A process's **service time** is the total number of CPU ticks it needs. Each tick of execution reduces its remaining service by one. Switching processes takes no time.

A ready process has arrived but is not running or finished. In the queue notation below, the leftmost process is next.

2. Scheduling rules

Rule 1. Join the back of the queue on arrival

New processes enter at the tail of the ready queue. Processes that arrive together enter in their input order. A process already waiting stays ahead of a new arrival.

For example, if B is waiting and C arrives, the queue becomes [B, C]. C does not move ahead because it has a shorter service time.

Rule 2. Run the process at the front

When the CPU is free, remove the queue head and run it. Service time and process ID do not affect this choice.

Rule 3. Let the running process finish

FCFS is **non-preemptive**: an arrival cannot take the CPU away from the running process. If A has four ticks left when a one-tick process B arrives, B waits for all four ticks.

Rule 4. Complete a process when its remaining service reaches zero

Remove the finished process before selecting another one. If A executes its final tick from time 4 to time 5, it completes at time 5. The next process may start at that same time.

Rule 5. Idle only when nothing is ready

If the ready queue is empty, wait for the next arrival. Stop when every process has finished. An empty queue does not interrupt a process already running.

3. Events at a tick boundary

At time t , first account for execution during the preceding tick. Then:

1. Remove the running process if it has finished.
2. Add all arrivals at t to the queue tail, in input order.
3. If the CPU is free, dispatch the queue head.

For example, suppose A finishes at time 5, B is waiting, and C arrives at time 5. The queue becomes [B, C], so B starts at time 5. There is no idle tick between A and B.

4. Worked example

The processes are entered in the order A, B, C. All times are in ticks.

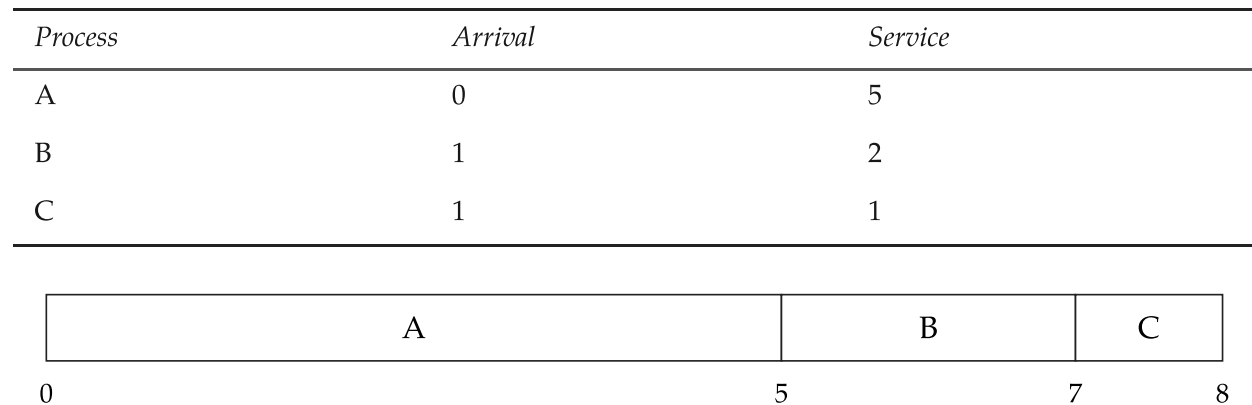


Figure 1. CPU schedule (time in ticks).

A starts at time 0. B and C arrive at time 1, but neither can interrupt it. At time 5, B runs before C because B was listed first among the simultaneous arrivals. The shorter process C still waits until time 7.